

Approach sheet

Market Basket Analysis with Python – Target

Objective: Build a comprehensive Customer Purchasing Behaviour and Product Recommendation Report using Python for data analysis. This report should clean and consolidate Target's transaction and customer data, apply techniques such as Market Basket Analysis, clustering, and association rule mining, and provide actionable insights for personalised product recommendations, promotional strategies, and inventory optimisation.

Task 1: Data Cleaning and Preparation

Task 1.1: Remove Duplicates or inconsistent survey responses.

- For this task, I used this code to find the duplicate values and found that there are no duplicates in the dataset.

```
: df.duplicated().sum()  
:  
: np.int64(0)
```

Task 1.2: Standardise categorical/Numerical entries (e.g., frequency levels, gender, recommendation responses).

- I first checked the data, which needs to be standardised like this.

```
: df['Search_Result_Exploration'].value_counts()  
:  
: Search_Result_Exploration  
: First page      418  
: Multiple pages  382  
: Name: count, dtype: int64  
:  
: df['Gender'].value_counts()  
:  
: Gender  
: Male      214  
: Female    201  
: Others     196  
: Prefer not to say 189  
: Name: count, dtype: int64
```

- I standardised the values in purchase frequency into weekly, monthly, monthly high and weekly high to understand it more clearly.

```
df['Purchase_Frequency'].value_counts()
```

```
Purchase_Frequency
Multiple times a week    173
Less than once a month  168
Once a month            164
Few times a month       153
Once a week             142
Name: count, dtype: int64
```

standardizing the frequency levels

```
df['Purchase_Frequency'] = df['Purchase_Frequency'].replace({
    'Once a week': 'weekly',
    'Multiple times a week': 'weekly_high',
    'Less than once a month': 'Quarterly',
    'Once a month': 'monthly',
    'Few times a month': 'monthly_high'
})
```

```
df['Purchase_Frequency'].value_counts()
```

```
Purchase_Frequency
weekly_high    173
Quarterly      168
monthly        164
monthly_high   153
weekly         142
Name: count, dtype: int64
```

- For the browsing frequency, I used High, low, moderate and rarely as the standard terms.

```
df['Browsing_Frequency'].value_counts()
```

```
Browsing_Frequency
Multiple times a day    207
Rarely                 200
Few times a month      197
Few times a week       196
Name: count, dtype: int64
```

Naming 'Multiple times a day' as 'High', 'Rarely' as 'Rarely', 'Few times a month' as 'low' and 'Few times a week' as 'moderate'.

```
df['Browsing_Frequency'] = df['Browsing_Frequency'].replace({
    'Multiple times a day': 'High',
    'Rarely': 'Rarely',
    'Few times a month': 'low',
    'Few times a week': 'moderate'
})
df['Browsing_Frequency'].value_counts()
```

```
Browsing_Frequency
High        207
Rarely      200
low         197
moderate    196
Name: count, dtype: int64
```

- The recommendation helpfulness variable was standardised into an ordinal numerical scale to ensure consistency and enable quantitative analysis.

```
df['Recommendation_Helpfulness'].value_counts()
```

```
Recommendation_Helpfulness
Sometimes    270
Yes          265
No           265
Name: count, dtype: int64
```

```
df['Recommendation_Helpfulness_Score'] = df['Recommendation_Helpfulness'].str.strip().str.lower()
mapping = {
    'yes': 1,
    'sometimes': 2,
    'no': 0
}
df['Recom_Helpfulness_Score'] = df['Recommendation_Helpfulness_Score'].map(mapping)
df['Recom_Helpfulness_Score'].value_counts()
```

```
Recom_Helpfulness_Score
2    270
1    265
0    265
Name: count, dtype: int64
```

Task 1.3: Handle missing values and inconsistent formats in Product_Search_Method and other fields.

- I checked for any missing values in the data and found 180 null values in the data.

```
df.isnull().sum()
```

```
Timestamp      0
age            0
Gender         0
Purchase_Frequency
Purchase_Categories
Personalized_Recommendation_Frequency
Browsing_Frequency
Product_Search_Method      180
Search_Result_Exploration
Customer_Reviews_Importance
Add_to_Cart_Browsing
Cart_Completion_Frequency
Cart_Abandonment_Factors
Saveforlater_Frequency
Review_Left
Review_Reliability
Review_Helpfulness
Personalized_Recommendation_Frequency
Recommendation_Helpfulness
Rating_Accuracy
Shopping_Satisfaction
Service_Appreciation
Improvement_Areas
transaction      0
dtype: int64
```

- I replaced the null values with mode value.

```
df['Product_Search_Method'].fillna(df['Product_Search_Method'].mode()[0], inplace=True)
```

C:\Users\alpan\AppData\Local\Temp\ipykernel_5336\3407113432.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Product_Search_Method'].fillna(df['Product_Search_Method'].mode()[0], inplace=True)
```

```
df.isnull().sum()
```

```
Timestamp      0
age            0
Gender         0
Purchase_Frequency
Purchase_Categories
Personalized_Recommendation_Frequency
Browsing_Frequency
Product_Search_Method
Search_Result_Exploration
Customer_Reviews_Importance
Add_to_Cart_Browsing
Cart_Completion_Frequency
Cart_Abandonment_Factors
Saveforlater_Frequency
Review_Left
Review_Reliability
Review_Helpfulness
Personalized_Recommendation_Frequency
Recommendation_Helpfulness
Rating_Accuracy
Shopping_Satisfaction
Service_Appreciation
Improvement_Areas
transaction      0
```

- In service appreciation, there is an inappropriate value found; I replaced it with the mode value.

```
: df['Service_Appreciation'].value_counts()
```

```
: Service_Appreciation
User-friendly website/app interface    105
Competitive prices                      100
Wide product selection                  98
Customer service                       90
Product recommendations                 85
.                                       85
All the above                          83
Customer service                       82
Quick delivery                         72
Name: count, dtype: int64
```

Service appreciation has some data with no name so replaced it with mode value

```
: df['Service_Appreciation'] = df['Service_Appreciation'].replace(df['Service_Appreciation'].mode())
df['Service_Appreciation'].value_counts()
```

```
: Service_Appreciation
User-friendly website/app interface    105
Competitive prices                      100
Wide product selection                  98
Customer service                       90
Product recommendations                 85
.                                       85
All the above                          83
Customer service                       82
Quick delivery                         72
Name: count, dtype: int64
```

Task 1.4: Rename duplicate or misformatted columns (e.g., remove trailing spaces in Rating_Accuracy).

- I renamed the columns which are not in standard form and reduced the length of names wherever possible.

```
df.columns=['Timestamp','Age','Gender','Purchase_freq','Purchase_cat',
            'Personalized_recom_freq(cat)','Browse_freq','Product_Search_Method',
            'Search_Result_Exploration','Customer_Reviews_Imp','Add_to_Cart_Brows',
            'Cart_Completion_Freq','Cart_Abandon_Factors','Save_for_later_Freq',
            'Review_Left','Review_Reliability','Review_Helpfulness','Personalized_Recom_Freq(num)',
            'Recom_Helpfulness','Rating_Accuracy','Shopping_Satisfaction','Service_Appreciation','Improvement_Areas','transaction',
            'Recommendation_Helpfulness_Score','Recom_Helpfulness_Score']
```

```
df.columns
```

```
Index(['Timestamp', 'Age', 'Gender', 'Purchase_freq', 'Purchase_cat',
      'Personalized_recom_freq(cat)', 'Browse_freq', 'Product_Search_Method',
      'Search_Result_Exploration', 'Customer_Reviews_Imp',
      'Add_to_Cart_Brows', 'Cart_Completion_Freq', 'Cart_Abandon_Factors',
      'Save_for_later_Freq', 'Review_Left', 'Review_Reliability',
      'Review_Helpfulness', 'Personalized_Recom_Freq(num)',
      'Recom_Helpfulness', 'Rating_Accuracy', 'Shopping_Satisfaction',
      'Service_Appreciation', 'Improvement_Areas', 'transaction',
      'Recommendation_Helpfulness_Score', 'Recom_Helpfulness_Score'],
      dtype='object')
```

Task 1.5: Convert numerical rating columns (e.g., Customer_Reviews_Importance, Shopping_Satisfaction) to appropriate numeric types for analysis.

- These columns were already in appropriate numerical datatypes.

```
df['Customer_Reviews_Imp'].value_counts()
```

```
Customer_Reviews_Imp
5    171
2    163
1    161
3    159
4    146
Name: count, dtype: int64
```

```
df['Shopping_Satisfaction'].value_counts()
```

```
Shopping_Satisfaction
1    173
2    167
5    166
3    165
4    129
Name: count, dtype: int64
```

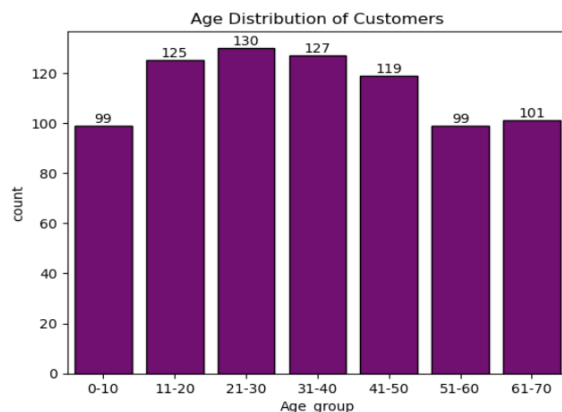
Task 2: Descriptive Behaviour Analysis

Task 2.1: Summarise customer demographics (age, gender distribution).

- **Age:** I summarized age distribution using descriptive statistics and grouped customers into age bins using `pd.cut` for better demographic analysis.

```
bins = [0,10,20,30,40,50,60,70]
age_labels=['0-10','11-20','21-30','31-40','41-50','51-60','61-70']
df['Age_group'] = pd.cut(df['Age'], bins=bins, labels=age_labels, right =True)

ag=sns.countplot(x='Age_group', data=df, order=age_labels, color='purple',edgecolor='black')
ag.bar_label(ag.containers[0])
plt.xlabel("Age_group")
plt.title("Age Distribution of Customers")
plt.xticks(rotation=0)
plt.show()
```



- It is indicated from the bar graph that the majority of customers are younger that is between 21-30 age group.
- **Gender:** I used count-based analysis to evaluate gender distribution, identifying which groups have higher representation and assessing whether the dataset is balanced or skewed.

```
df['Gender'].value_counts()
```

```
Gender
Male      214
Female    201
Others     196
Prefer not to say 189
Name: count, dtype: int64
```

```
df['Gender'].value_counts(normalize=True) * 100
```

```
Gender
Male      26.750
Female    25.125
Others     24.500
Prefer not to say 23.625
Name: proportion, dtype: float64
```

```

: gd=df['Gender'].value_counts().plot(kind='bar', title='Gender Distribution', color='purple',edgecolor='black')
gd.bar_label(gd.containers[0])
plt.xticks(rotation=0)
plt.show()

```



- The gender distribution shows a higher number of male customers compared to female and other categories, indicating a slight dominance of male users in the dataset.
- Then we check for any outliers in the numerical data and checked for distribution of data in the categorical columns using boxplot and countplot.
- Remove unnecessary columns which are not required in the analysis.
- I removed Timestamp, Transaction.

```

: df.drop(columns=['Timestamp','transaction'], inplace=True)

```

```

: df.columns

```

```

: Index(['Age', 'Gender', 'Purchase_freq', 'Purchase_cat',
       'Personalized_recom_freq(cat)', 'Browse_freq', 'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Imp',
       'Add_to_Cart_Brows', 'Cart_Completion_Freq', 'Cart_Abandon_Factors',
       'Save_for_later_Freq', 'Review_Left', 'Review_Reliability',
       'Review_Helpfulness', 'Personalized_Recom_Freq(num)',
       'Recom_Helpfulness', 'Rating_Accuracy', 'Shopping_Satisfaction',
       'Service_Appreciation', 'Improvement_Areas',
       'Recommendation_Helpfulness_Score', 'Recom_Helpfulness_Score',
       'Age_group'],
      dtype='object')

```

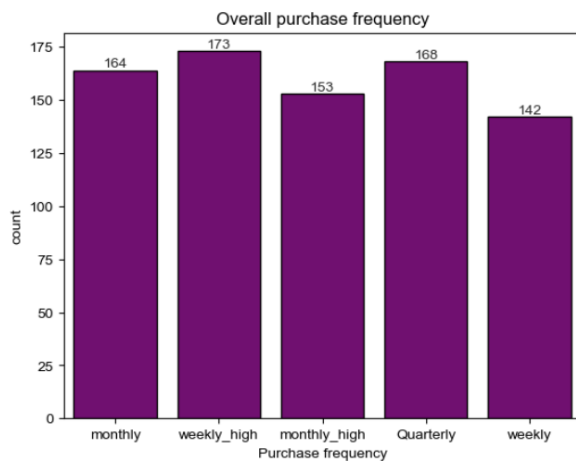
Task 2.2: Analyze overall purchase frequency and most popular product categories.

- For this task I created a count plot to know about the distribution of the values.

```

pf=sns.countplot(x='Purchase_freq', data=df, color = 'purple', edgecolor = 'black')
plt.xlabel('Purchase frequency')
plt.title('Overall purchase frequency')
sns.set_style("whitegrid")
plt.figure(figsize=(8,2))
pf.bar_label(pf.containers[0])
plt.show()

```



<Figure size 800x200 with 0 Axes>

--> It is clearly visible that weekly_high that is multiple times a week products are purchased.

- For the purchase category, there are multiple categories in a single purchase so I split multiple product categories stored in a single column into individual entries using explode, allowing me to accurately count and visualize the most popular product categories.

```

# For purchase category
df['Purchase_cat'].value_counts()

```

Purchase_cat	
Groceries and Gourmet Food;Beauty and Personal Care;others	37
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;Home and Kitchen;others	34
Home and Kitchen	33
Groceries and Gourmet Food;Beauty and Personal Care	32
Groceries and Gourmet Food;Clothing and Fashion;others	32
Beauty and Personal Care;Clothing and Fashion	32
Beauty and Personal Care;Home and Kitchen	31
Groceries and Gourmet Food	31
others	30
Beauty and Personal Care;Clothing and Fashion;Home and Kitchen;others	29
Clothing and Fashion;others	29
Beauty and Personal Care;others	28
Groceries and Gourmet Food;Home and Kitchen	28
Groceries and Gourmet Food;Beauty and Personal Care;Home and Kitchen	28
Groceries and Gourmet Food;Clothing and Fashion;Home and Kitchen	28
Groceries and Gourmet Food;Clothing and Fashion;Home and Kitchen;others	28
Beauty and Personal Care	27
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion	26
Groceries and Gourmet Food;Home and Kitchen;others	26
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;Home and Kitchen	26
Beauty and Personal Care;Home and Kitchen;others	25
Clothing and Fashion;Home and Kitchen	25
Beauty and Personal Care;Clothing and Fashion;Home and Kitchen	25
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;others	24
Groceries and Gourmet Food;Clothing and Fashion	23
Beauty and Personal Care;Clothing and Fashion;others	22
Clothing and Fashion	21
Home and Kitchen;others	20
Clothing and Fashion;Home and Kitchen;others	20

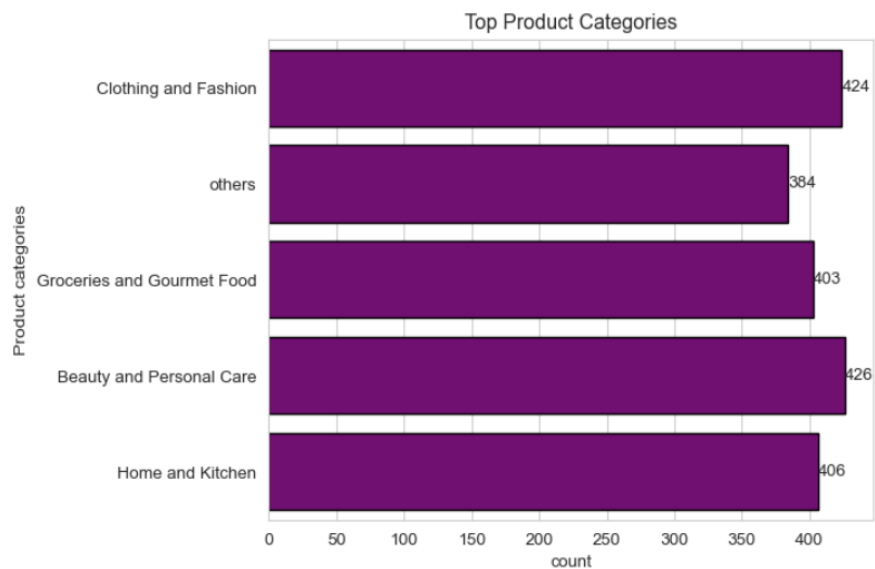
```

df_splitted = df.assign(Purchase_cat=df['Purchase_cat'].str.split(';')).explode('Purchase_cat')
df_splitted['Purchase_cat'].value_counts()

```

Purchase_cat	
Beauty and Personal Care	426
Clothing and Fashion	424
Home and Kitchen	406
Groceries and Gourmet Food	403
others	384
Name: count, dtype: int64	

```
pc=sns.countplot(y='Purchase_cat',data=dfSplitted, color='purple', edgecolor='black')
pc.bar_label(pc.containers[0])
plt.ylabel('Product categories')
plt.title("Top Product Categories")
plt.show()
```

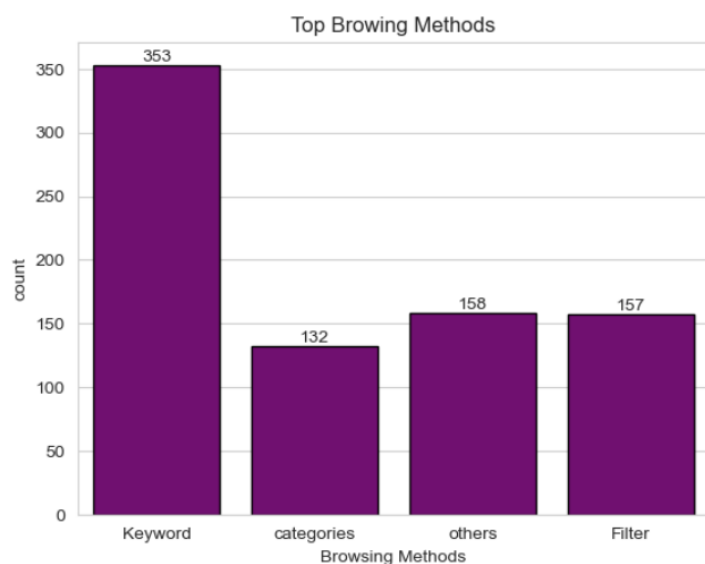


--> Beauty and Personal care followed by Clothing and fashion are the most frequently purchased categories, indicating strong customer interest in these segments.

Task 2.3: Identify top browsing methods and most common cart abandonment factors.

- Countplot created for identifying top browsing methods

```
psm=sns.countplot(x='Product_Search_Method', data=df, color = 'purple', edgecolor = 'black')
plt.xlabel('Browsing Methods')
plt.title('Top Browsing Methods')
sns.set_style("whitegrid")
plt.figure(figsize=(8,5))
psm.bar_label(psm.containers[0])
plt.show()
```



<Figure size 800x500 with 0 Axes>

--> Keyword is the most used browsing method.

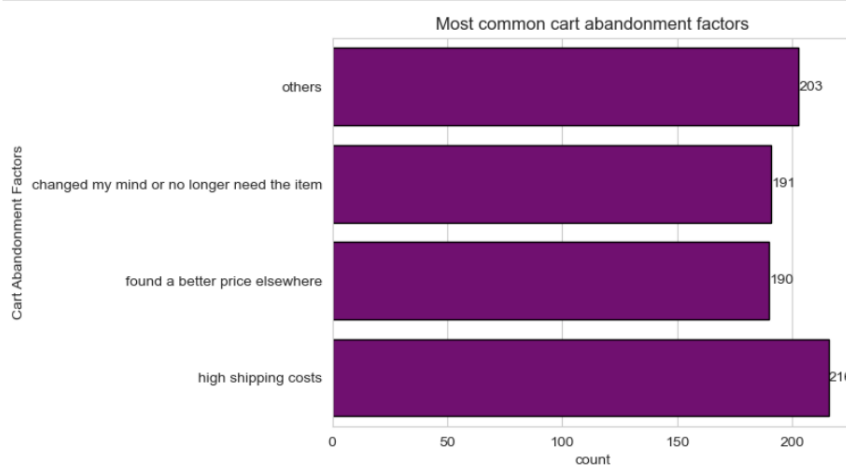
- There were spaces in the data so spaces were removed using strip and sentence were converted to lowercase for uniformity
- Countplot created to identify the frequent reasons for cart abandonment.

```
df['Cart_Abandon_Factors'].value_counts()
```

```
Cart_Abandon_Factors
High shipping costs    216
others                 203
Changed my mind or no longer need the item    191
Found a better price elsewhere    190
Name: count, dtype: int64
```

```
df['Cart_Abandon_Factors'] = df['Cart_Abandon_Factors'].str.strip().str.lower()
```

```
caf=sns.countplot(y='Cart_Abandon_Factors', data=df, color = 'purple', edgecolor = 'black')
plt.ylabel('Cart Abandonment Factors')
plt.title('Most common cart abandonment factors')
plt.xticks(rotation=0)
sns.set_style("whitegrid")
caf.bar_label(caf.containers[0])
plt.show()
```



--> High shipping costs is the main factor for cart abandonment.

Task 2.4: Calculate mean and median satisfaction, recommendation helpfulness, and rating accuracy

- Mean and median for satisfaction and rating accuracy were calculated by .mean() and .median() but for recommendation helpfulness it is not possible as this is a categorical column so I calculated mode by .mode().

```
df[['Shopping_Satisfaction', 'Recom_Helpfulness', 'Rating_Accuracy']].dtypes
```

```
Shopping_Satisfaction    int64
Recom_Helpfulness        object
Rating_Accuracy          int64
dtype: object
```

```
df['Recom_Helpfulness'].mode()
```

```
0    Sometimes
Name: Recom_Helpfulness, dtype: object
```

```
df[['Shopping_Satisfaction',
    'Rating_Accuracy']].agg(['mean', 'median'])
```

	Shopping_Satisfaction	Rating_Accuracy
mean	2.935	3.05875
median	3.000	3.00000

Task 2.5: Generate summary statistics and visualisations for key behavioural variables.

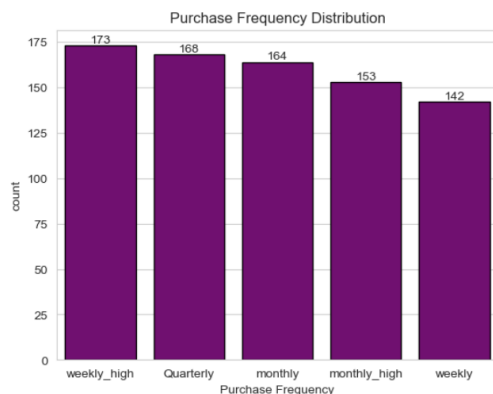
- Summary statistics were generated for key behavioral variables to understand central tendency and variability in customer purchase frequency, satisfaction, and engagement patterns.

```
behavior_cols = df[['Purchase_freq_score', 'Personalized_Recom_Freq(num)', 'Rating_Accuracy', 'Shopping_Satisfaction']].describe()
behavior_cols
```

	Purchase_freq_score	Personalized_Recom_Freq(num)	Rating_Accuracy	Shopping_Satisfaction
count	800.000000	800.000000	800.000000	800.000000
mean	3.015000	2.981250	3.058750	2.935000
std	1.445644	1.450354	1.439753	1.436438
min	1.000000	1.000000	1.000000	1.000000
25%	2.000000	2.000000	2.000000	2.000000
50%	3.000000	3.000000	3.000000	3.000000
75%	4.000000	4.000000	4.000000	4.000000
max	5.000000	5.000000	5.000000	5.000000

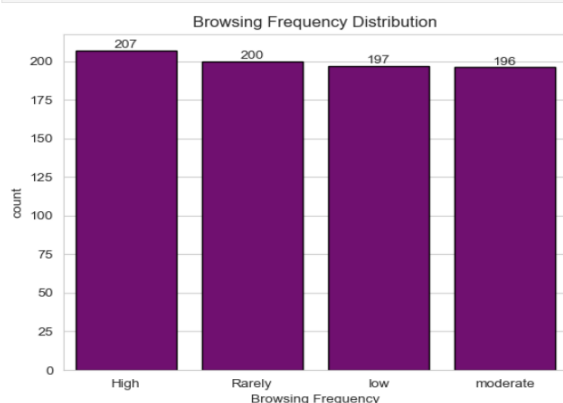
- Visualisations: I created few visuals for key behavioural variables using countplot from seaborn.
- Purchase Frequency and Browsing Frequency Distribution: Using Countplot.

```
ab=sns.countplot(x='Purchase_freq', data=df, order=df['Purchase_freq'].value_counts().index,color = 'purple', edgecolor='black')
ab.bar_label(ab.containers[0])
plt.xlabel('Purchase Frequency')
plt.title("Purchase Frequency Distribution")
plt.xticks(rotation=0)
plt.show()
```



--> Most customers fall under weekly_high (173), which indicates high engagement, weekly(142) is lowest that is fewer low-frequency users. Overall, users are skewed toward higher purchase frequency, suggesting strong customer activity.

```
ac=sns.countplot(x='Browse_freq', data=df, order=df['Browse_freq'].value_counts().index,color = 'purple', edgecolor='black')
ac.bar_label(ac.containers[0])
plt.xlabel('Browsing Frequency')
plt.title("Browsing Frequency Distribution")
plt.xticks(rotation=0)
plt.show()
```

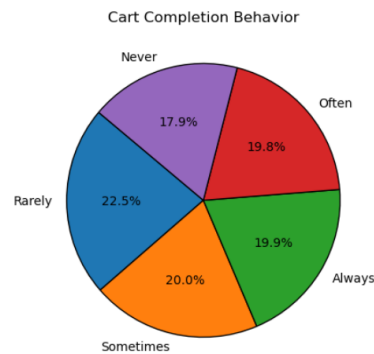


--> Customers which are browsing daily are highest (207) in number and customers who browse few times in a week or in a month are lowest around 197 and 196.

- Cart Completion Frequency: using Piechart to see the proportion of how many customers complete their cart.

I used a pie chart to represent the distribution of cart completion frequency, making it easier to interpret proportional differences across categories.

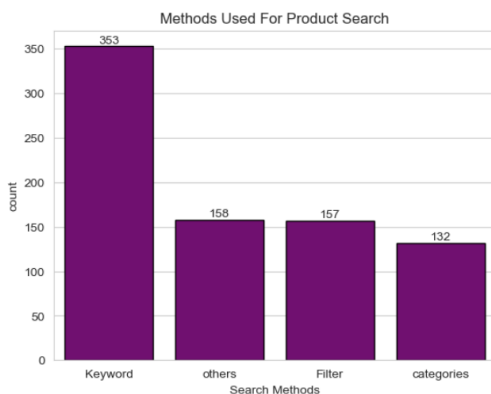
```
cart_counts=df['Cart_Completion_Freq'].value_counts()
plt.figure(figsize=(5,5))
plt.pie(cart_counts,labels=cart_counts.index,autopct='%1.1f%%',startangle=140,wedgeprops={'edgecolor': 'black'})
plt.title("Cart Completion Behavior")
plt.show()
```



--> A significant portion of customers complete purchases frequently, while a noticeable segment shows lower completion rates, indicating potential drop-offs in the purchase funnel.

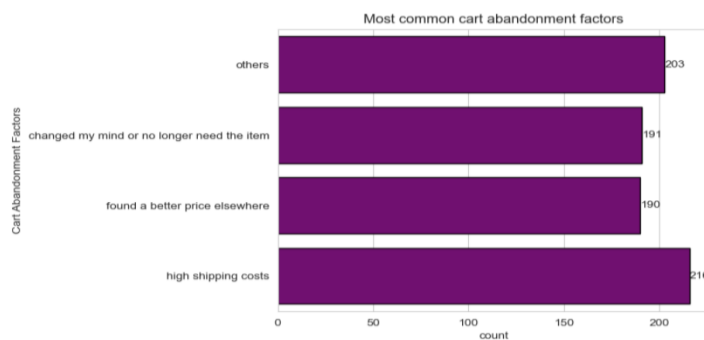
- Methods used for product search: Using countplot

```
ae=sns.countplot(x="Product_Search_Method", data=df, color = 'purple', edgecolor = 'black', order=df['Product_Search_Method'].value_counts().index)
ae.bar_label(ae.containers[0])
plt.xlabel("Search Methods")
plt.title("Methods Used For Product Search")
plt.show()
```



--> Keyword method (353) is used most frequent than any other search methods and categories being the lowest (132).

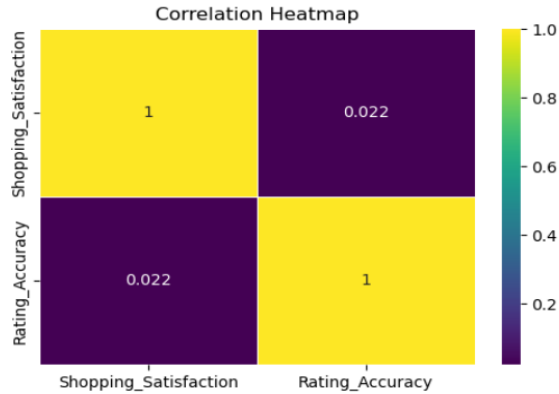
- Most common cart abandonment factor.



--> Most of the customers abandon the cart due to its high shipping costs and finding a better price elsewhere is the least cause for cart abandonment.

- Heatmap is created for the Shopping satisfaction and rating accuracy to analyse the correlation between the two.

```
data1 = df[['Shopping_Satisfaction', 'Rating_Accuracy']]
corr = data1.corr()
plt.figure(figsize=(6,4))
sns.heatmap(corr, annot=True, cmap='viridis', linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```



--> There is no relationship between Shopping satisfaction and rating accuracy as the value is near to 0.

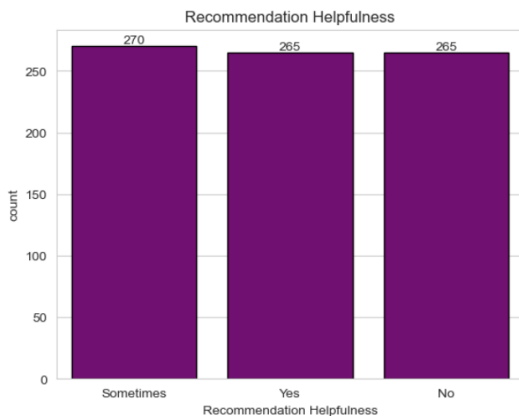
- Boxplot for shopping and rating accuracy distribution.

```
sns.boxplot(data=df[['Shopping_Satisfaction', 'Rating_Accuracy']])
plt.title("Customer Experience Metrics")
plt.show()
```



--> Both satisfaction and rating accuracy show similar distributions, indicating consistent customer perception across experience and trust metrics.

- Distribution of Recommendation helpfulness.

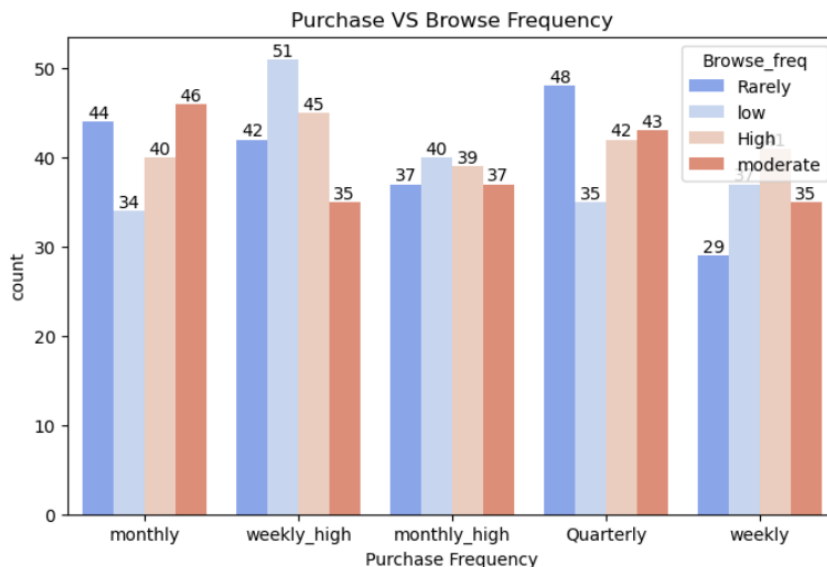


--> Almost equal distribution of recommendation helpfulness that is recommendation are not really effective.

--> Recommendation helpfulness is evenly distributed, indicating that the current recommendation system lacks strong impact and needs optimization.

- Purchase VS Browsing Frequency : Using Countplot to check for engagement.

```
plt.figure(figsize=(8,5))
pvb=sns.countplot(x='Purchase_freq', hue='Browse_freq',data =df, palette='coolwarm')
for container in pvb.containers:
    pvb.bar_label(container)
plt.xlabel('Purchase Frequency')
plt.title('Purchase VS Browse Frequency')
plt.show()
```



--> Users with higher browsing → slightly higher purchase activity

--> But pattern is not very strong

--> Many users browse frequently but don't convert consistently

Task 3: Customer Segmentation and Profiling

Task 3.1: Segment customers based on purchase frequency and shopping satisfaction levels.

- I segmented customers as: Firstly created a numerical column for purchase frequency to give scores to customers on the basis of purchase frequency.
- Then I segmented customers by their purchase frequency score and shopping satisfaction.

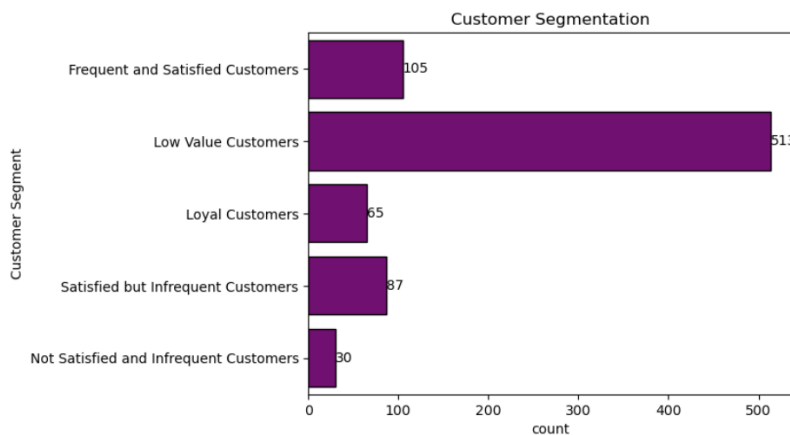
```
freq_map = {
    'weekly_high': 5,
    'weekly': 4,
    'monthly_high': 3,
    'monthly': 2,
    'Quarterly': 1
}

df['Purchase_freq_score'] = df['Purchase_freq'].map(freq_map)

def segment(row):
    if row['Purchase_freq_score'] == 5 and row['Shopping_Satisfaction'] >= 4:
        return 'Loyal Customers'
    elif row['Purchase_freq_score'] == 4 and row['Shopping_Satisfaction'] < 4:
        return 'Frequent and Satisfied Customers'
    elif row['Purchase_freq_score'] == 3 and row['Shopping_Satisfaction'] >= 3:
        return 'Satisfied but Infrequent Customers'
    elif row['Purchase_freq_score'] == 2 and row['Shopping_Satisfaction'] < 2:
        return 'Not Satisfied and Infrequent Customers'
    else:
        return 'Low Value Customers'

df['Customer_Segment'] = df.apply(segment, axis=1)
```

- Then by using countplot I visualized the distribution of the customers segments.



--> The majority of customers fall under the low-value segment, indicating low engagement levels.

--> However, a significant portion of satisfied but infrequent customers presents an opportunity for conversion into loyal customers through targeted marketing strategies.

--> Additionally, the small proportion of dissatisfied customers highlights areas for service improvement to reduce churn.

Task 3.2: Create profiles such as:

- **Frequent Buyers: High purchase frequency, high satisfaction.**
- **Occasional Shoppers: Medium frequency, moderate satisfaction.**
- **At-Risk Customers: Low satisfaction or frequent cart abandonment.**

- I created profiles for frequent Buyers , Occasional Shoppers and At risk customers on the basis of their purchase frequency, Shopping satisfaction and Cart Abandon score.
- The threshold was decided using the statistical distribution summary.

```
df['Cart_Abandon_Factors'].value_counts()
```

```
Cart_Abandon_Factors
high shipping costs    216
others                203
changed my mind or no longer need the item    191
found a better price elsewhere    190
Name: count, dtype: int64
```

```
cart_map={
    "high shipping costs": 1,
    "others":2,
    "changed my mind or no longer need the item":3,
    "found a better price elsewhere":4
}
df['cart_abandon_score'] = df['Cart_Abandon_Factors'].map(cart_map)
```

```
df[['Purchase_freq_score', 'Shopping_Satisfaction', 'cart_abandon_score']].describe()
```

	Purchase_freq_score	Shopping_Satisfaction	cart_abandon_score
count	800.000000	800.000000	203.0
mean	2.985000	2.935000	2.0
std	1.445644	1.436438	0.0
min	1.000000	1.000000	2.0
25%	2.000000	2.000000	2.0
50%	3.000000	3.000000	2.0
75%	4.000000	4.000000	2.0
max	5.000000	5.000000	2.0

--> From the Data distribution, we decide the thresholds as median is nearly equal to 3, where values ≥ 4 represents high behaviour and ≤ 2 represents low behaviour.

Frequent Buyers: High purchase frequency ≥ 4 , high satisfaction ≥ 4 .

Occasional Shoppers: Medium frequency is in between 2 and 4, moderate satisfaction is in between 2 and 4.

At-Risk Customers: Low satisfaction ≤ 2 or frequent cart abandonment ≥ 1 .

```
] def profiles (row):
    if row['Shopping_Satisfaction'] <=2 or row['cart_abandon_score'] == 1:
        return 'At-Risk Customers'
    elif row['Purchase_freq_score'] >=4 and row['Shopping_Satisfaction'] >=4:
        return 'Frequent Buyers'
    elif (2 < row['Purchase_freq_score'] < 4) and (2 < row['Shopping_Satisfaction'] < 4):
        return 'Occasional Buyers'
    else:
        return 'Others'
df['Customer_profiles'] = df.apply(profiles, axis=1)

] df['Customer_profiles'].value_counts()

] Customer_profiles
At-Risk Customers    340
Others              320
Frequent Buyers     112
Occasional Buyers    28
Name: count, dtype: int64

] cp=sns.countplot(x='Customer_profiles', data=df, color='purple',edgecolor='black')
cp.bar_label(cp.containers[0])
plt.xlabel('Customer_profiles')
plt.title('Customer_profiles_segmentation')
plt.show()
```



--> Majority of customers are at risk (340) and very few are occasional buyers (28), frequent buyers are also very moderate(112).

Task 3.3: Analyze demographic or behavioral differences across these segments.

- I created a boxplot to check the Age distribution of customers across the segments.
- Firstly checked the statistical summary of the segments, then created the boxplot.
- Age Distribution across Customer profiles

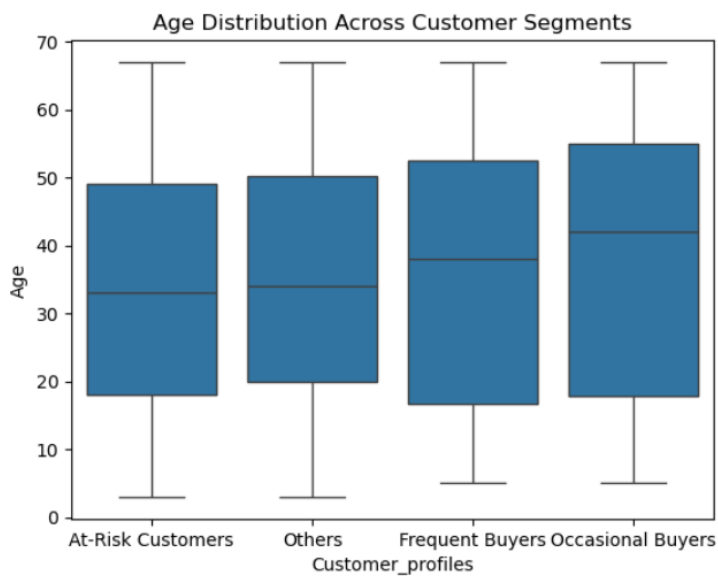
```
] df.groupby('Customer_profiles')['Age'].mean()

] Customer_profiles
At-Risk Customers    33.705882
Frequent Buyers     35.892857
Occasional Buyers    37.607143
Others              35.178125
Name: Age, dtype: float64

] df.groupby('Customer_profiles')['Age'].describe()

]
count    mean    std  min   25%   50%   75%  max
Customer_profiles
At-Risk Customers    340.0  33.705882  18.826846  3.0  18.00  33.0  49.00  67.0
Frequent Buyers     112.0  35.892857  19.951905  5.0  16.75  38.0  52.50  67.0
Occasional Buyers     28.0  37.607143  19.604118  5.0  17.75  42.0  55.00  67.0
Others              320.0  35.178125  18.657350  3.0  20.00  34.0  50.25  67.0

] sns.boxplot(x='Customer_profiles', y='Age', data=df)
plt.title("Age Distribution Across Customer Segments")
plt.show()
```



--> Occasional buyers tend to be more mature customers who shop less frequently but may have higher purchasing power.

--> Frequent buyers are younger to middle-aged, showing active engagement with the platform.

--> Younger users are more likely to be at risk — possibly due to: price and alternatives.

- Gender distribution across Customer profiles
- I grouped the customers by gender and profiles.

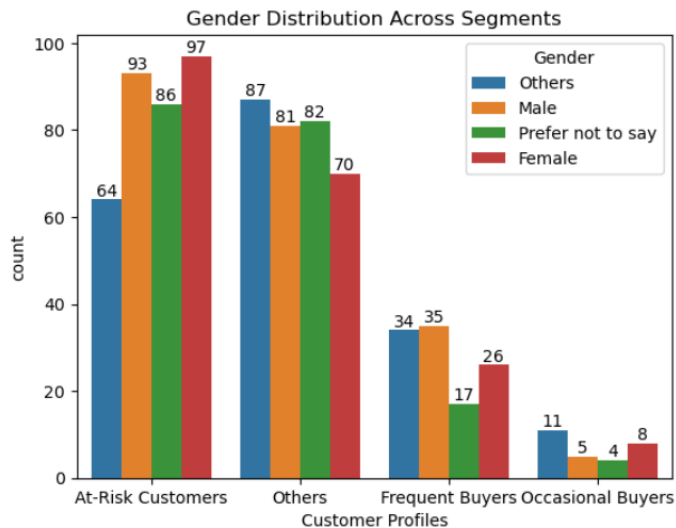
```
pd.crosstab(df['Customer_profiles'], df['Gender'])
```

	Gender	Female	Male	Others	Prefer not to say
Customer_profiles					
At-Risk Customers		97	93	64	86
Frequent Buyers		26	35	34	17
Occasional Buyers		8	5	11	4
Others		70	81	87	82

```
pd.crosstab(df['Customer_profiles'], df['Gender'], normalize='index') * 100
```

	Gender	Female	Male	Others	Prefer not to say
Customer_profiles					
At-Risk Customers		28.529412	27.352941	18.823529	25.294118
Frequent Buyers		23.214286	31.250000	30.357143	15.178571
Occasional Buyers		28.571429	17.857143	39.285714	14.285714
Others		21.875000	25.312500	27.187500	25.625000

```
cpg=sns.countplot(x='Customer_profiles', hue='Gender', data=df)
for container in cpg.containers:
    cpg.bar_label(container)
plt.xlabel('Customer Profiles')
plt.title("Gender Distribution Across Segments")
plt.xticks(rotation=0)
plt.show()
```



--> Customer risk (low satisfaction/abandonment) is not gender-specific — it's a platform-wide issue.

--> Frequent Buyers are Balanced Across Genders - Loyal customers exist in all gender groups equally

--> Occasional Buyers are Very Low in Every Gender

--> Others segments are evenly distributed

"High at-risk across all genders → poor customer experience consistency" "Low occasional buyers → weak engagement funnel" "Balanced frequent buyers → broad appeal of platform"

Task 3.4: Use clustering (e.g., K-Means) for behavioral grouping based on survey responses.

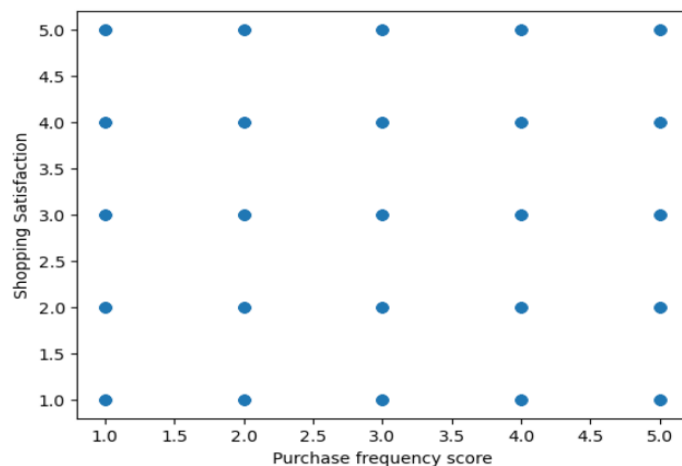
- I performed clustering on two behavioural columns Purchase_freq_score and Shopping_Satisfaction defined x = Purchase_freq_score and Shopping_Satisfaction.
- Checked for null values in columns and created a scatter plot to see the relationship between variables but I found no relationship between the variables so I go with these two columns.

```
features = ['Purchase_freq_score', 'Shopping_Satisfaction']
x=df[features]
```

```
df[features].isnull().sum()
```

```
Purchase_freq_score    0
Shopping_Satisfaction    0
dtype: int64
```

```
plt.scatter(x['Purchase_freq_score'],x['Shopping_Satisfaction'])
plt.xlabel('Purchase frequency score')
plt.ylabel('Shopping Satisfaction')
plt.show()
```



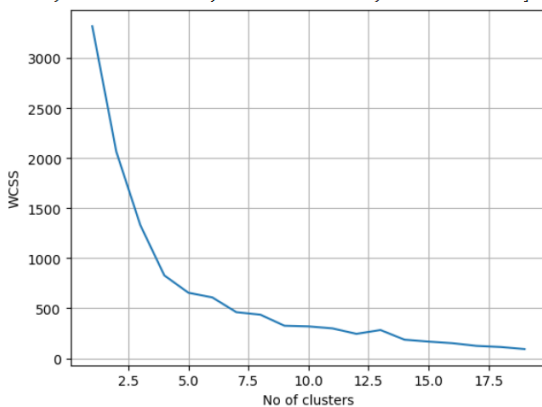
- Then from sklearn.cluster I imported KMeans and from sklearn.metrics I imported silhouette_score
- Then created 19 clusters randomly and created elbow plot to check how many clusters to be created for evaluation.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# wcss = within cluster sum of square
wcss = []
for i in range(1,20): # i = no of clusters,i=1,2,3,...19
    m = KMeans(n_clusters=i)
    m.fit(x)
    wcss.append(m.inertia_)

# Elbow plot between - x axis: no. of Ks(19), y axis: no. of WCSS values
print('WCSS\n',wcss)
plt.plot(list(range(1,20)),wcss)
plt.xlabel('No of clusters')
plt.ylabel('WCSS')
plt.grid()
plt.show()
```

WCSS

[3318.4399999999987, 2065.6697266421843, 1331.3250094406035, 828.4194733520536, 655.665104737077, 608.2955042124111, 460.93109460112424, 435.871335362536, 325.6662831122135, 318.83024694839054, 299.1233578022826, 244.41879871029693, 282.81090131895655, 186.05351873451497, 167.21426740532996, 151.0676902861813, 124.2671371659963, 113.22973363394891, 92.4027297394095]



```
for i in range(2,20): # i = no of clusters
    m = KMeans(n_clusters=i).fit(x)
    print(f'Silhouette Score at K={i}', silhouette_score(x,m.labels_))
```

Silhouette Score at K=2 0.36938755232634984
 Silhouette Score at K=3 0.4162141485610771
 Silhouette Score at K=4 0.4403572634295457
 Silhouette Score at K=5 0.43246145585751705
 Silhouette Score at K=6 0.3761747541611995
 Silhouette Score at K=7 0.45603868952769416
 Silhouette Score at K=8 0.4277179522537969
 Silhouette Score at K=9 0.5026274191452355
 Silhouette Score at K=10 0.4822093538299045
 Silhouette Score at K=11 0.5310460419166476
 Silhouette Score at K=12 0.5537817556035645
 Silhouette Score at K=13 0.5845696222929444
 Silhouette Score at K=14 0.6066865001631825
 Silhouette Score at K=15 0.6187905232346508
 Silhouette Score at K=16 0.6608750751057352
 Silhouette Score at K=17 0.7102382349425532
 Silhouette Score at K=18 0.7443157576061011
 Silhouette Score at K=19 0.7731938215613706

--> Although silhouette score increased with higher K values, I avoided overfitting by selecting an optimal K in the range of 3–8, balancing cluster quality with interpretability

- I chose to create 5 clusters as the silhouette score is increasing with higher k values.

```
k5 = KMeans(n_clusters=5)
k5.fit(x)
ypred5 = k5.predict(x)
print(ypred5)
```

```
[1 0 3 0 4 1 3 0 0 4 1 3 1 0 2 4 0 4 0 3 2 4 2 4 3 4 4 2 4 1 1 4 0 2 4 1 0
 3 1 3 4 1 2 3 4 4 4 3 2 2 1 3 0 1 4 1 1 3 4 2 2 1 4 3 0 2 3 1 1 1 3 3 2 4
 4 1 0 0 4 0 4 3 3 2 4 3 3 4 0 4 1 1 4 4 4 3 3 4 1 3 0 2 1 3 4 2 1 1 4 1 2
 1 0 4 1 4 3 4 0 1 3 1 2 4 0 3 1 3 3 2 1 1 3 1 1 1 0 4 3 0 3 1 4 0 4 2 2 1
 2 2 1 0 3 3 2 0 3 2 1 1 3 4 2 3 0 3 1 1 1 4 2 4 0 1 3 1 3 3 0 2 3 0 0 1 1
 3 1 1 4 0 3 4 2 3 1 2 2 4 1 3 2 2 0 0 1 3 1 3 0 1 1 1 3 4 0 1 2 3 1 0 3 4
 3 0 1 0 4 0 0 1 3 3 2 0 2 0 4 3 4 1 3 0 1 2 1 1 2 3 1 1 1 0 1 4 3 1 2 3 2
 3 4 1 4 4 2 2 1 0 3 2 3 3 3 0 3 0 1 4 1 4 0 1 1 1 1 3 1 3 2 3 1 2 0 2 1 3
 0 0 0 3 1 1 2 1 3 1 3 2 0 0 4 4 0 3 0 0 0 2 4 1 1 4 0 4 1 3 3 3 1 1 2 0 4
 0 2 1 0 4 3 0 4 3 0 4 1 1 0 4 2 3 1 2 2 2 1 4 1 2 2 3 2 2 3 4 3 4 1 3 4 4
 1 4 0 0 4 3 1 4 1 2 3 4 0 3 0 4 1 3 4 3 4 1 0 3 1 4 0 4 3 0 3 4 3 0 4 0 1
 1 0 3 2 3 0 1 0 1 4 1 4 4 0 1 1 1 4 1 1 3 1 3 3 0 1 1 1 1 2 1 4 4 3 1 4 4
 2 1 3 2 2 1 4 3 1 2 2 2 1 3 2 1 3 2 2 0 1 0 1 4 4 2 2 0 2 1 1 2 3 3 4 2 3
 1 3 3 1 4 1 1 1 2 4 1 0 3 0 3 4 1 0 2 0 1 2 2 2 1 1 3 4 3 4 1 3 3 1 2 1 0
 1 2 4 0 2 1 3 4 4 1 3 1 1 3 0 4 4 4 1 2 4 0 1 0 0 2 4 2 1 0 2 0 4 0 4 2 0
 3 4 1 0 1 2 1 1 2 1 4 3 1 4 3 1 1 3 1 3 1 0 4 1 0 4 0 0 4 3 3 1 3 2 0 1 1
 1 3 4 2 2 2 2 2 1 3 3 2 3 4 1 1 1 0 0 1 3 4 4 2 2 2 4 1 0 1 2 3 3 3 3 1
 1 1 3 4 4 2 1 4 2 0 2 3 4 3 4 1 2 1 1 4 0 3 4 4 4 1 0 2 3 2 0 0 3 1 4 0 4
 1 2 2 4 3 3 1 0 4 0 1 3 2 1 4 1 3 3 0 3 2 1 3 0 0 1 4 1 1 1 2 1 1 0 1 1 3
 2 2 0 3 4 3 1 1 4 4 1 2 0 1 1 1 4 4 3 0 1 1 1 1 3 1 3 1 4 1 2 3 1 2 2 2 4
 1 4 0 1 1 1 1 1 3 0 4 4 1 0 3 1 0 2 1 3 1 1 1 2 1 1 1 0 1 3 1 1 2 0 1 1 3
 2 1 3 0 1 1 1 3 0 0 4 3 4 3 1 2 3 1 4 1 1 3 3]
```

```
from collections import Counter
c1 = Counter(ypred5)
print(c1)
```

```
Counter({np.int32(4): 183, np.int32(2): 179, np.int32(0): 155, np.int32(1): 147, np.int32(3): 136})
```

```
: cent5 = k5.cluster_centers_
print(cent5)
```

```
[[2.60645161 1.73548387]
 [1.29931973 2.23129252]
 [4.54748603 3.96648045]
 [4.55147059 1.5       ]
 [1.96721311 4.57377049]]
```

```
: df['Cls5'] = ypred5
```

A new column was created in the data set, which indicates which customer will come in which group based on purchase frequency score and shopping satisfaction

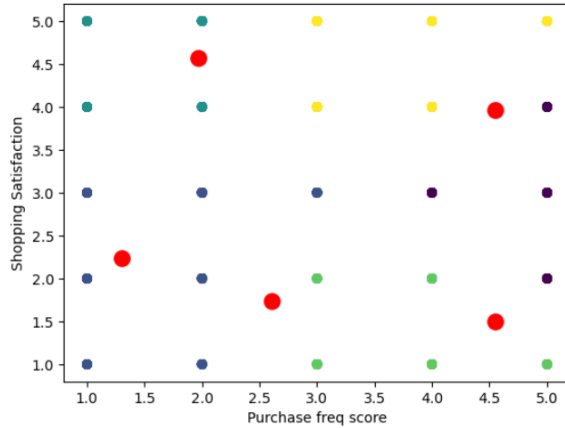
```
: cent5 = k5.cluster_centers_
print(cent5)
```

```
[[2.60645161 1.73548387]
 [1.29931973 2.23129252]
 [4.54748603 3.96648045]
 [4.55147059 1.5       ]
 [1.96721311 4.57377049]]
```

Then created a scatter plot to visualize the position of mean value of the cluster.

```
plt.scatter(x['Purchase_freq_score'],x['Shopping_Satisfaction'],c=df['Cls5'])
plt.scatter(cent5[:,0],cent5[:,1],color='red',s=120)
plt.title("Customer Clusters: Purchase Frequency Score vs Shopping Satisfaction")
plt.xlabel('Purchase freq score')
plt.ylabel('Shopping Satisfaction')
plt.show()
```

Customer Clusters: Purchase Frequency Score vs Shopping Satisfaction



This scatter plot visualizes customer segments formed using K-Means clustering, based on purchase frequency and satisfaction, with centroids representing the average behavior of each cluster.

- Then I did the cluster profiling , calculated average behaviour per cluster, and the overall average of clusters then combined both the results

```
res = df[features].groupby(df['Cls5']).mean()
res
```

	Purchase_freq_score	Shopping_Satisfaction
Cls5		
0	2.606452	1.735484
1	1.299320	2.231293
2	4.547486	3.966480
3	4.551471	1.500000
4	1.967213	4.573770

```
res1 = pd.DataFrame({'Purchase_freq_score':df['Purchase_freq_score'].mean(),
                    'Shopping_Satisfaction':df['Shopping_Satisfaction'].mean()},index=['Avg'])
res1
```

	Purchase_freq_score	Shopping_Satisfaction
Avg	2.985	2.935

- From the average value it is decided on the high and low value and then the conclusion is provided.

```
profiling = pd.concat([res1,res])
profiling
```

	Purchase_freq_score	Shopping_Satisfaction
Avg	2.985000	2.935000
0	2.880952	2.801587
1	1.393333	4.333333
2	1.798883	1.413408
3	4.005848	4.526316
4	4.649425	1.827586

--> Conclusion : Cluster profiling revealed distinct customer segments based on purchase behavior and satisfaction levels, helping identify loyal customers, at-risk users, and potential growth segments.

cls0 - Low Purchase_freq_score; High Shopping_Satisfaction

cls1 - High Purchase_freq_score; High Shopping_Satisfaction

cls2 - Low Purchase_freq_score; Low Shopping_Satisfaction

cls3 - High Purchase_freq_score; Low Shopping_Satisfaction

cls4 - High Purchase_freq_score; High Shopping_Satisfaction

Task 4: Recommendation and Review Insights

Task 4.1: Examine the relationship between recommendation helpfulness and shopping satisfaction.

- Firstly I changed the string value of Recommendation helpfulness to numeric value and then grouped them by mean value.
- Then I look for relationship between the two columns.

```
df['Recom_Helpfulness'].unique()

array(['Sometimes', 'Yes', 'No'], dtype=object)

df['Recommendation_Helpfulness_Score'] = df['Recom_Helpfulness'].str.strip().str.lower()
mapping = {
    'yes': 1,
    'sometimes': 2,
    'no': 0
}
df['Recom_Helpfulness_Score'] = df['Recommendation_Helpfulness_Score'].map(mapping)

df['Recom_Helpfulness_Score'].value_counts()

Recom_Helpfulness_Score
2    270
1    265
0    265
Name: count, dtype: int64

df['Recom_Helpfulness_Score'].isnull().sum()

np.int64(0)

df.groupby('Recom_Helpfulness_Score')['Shopping_Satisfaction'].mean()

Recom_Helpfulness_Score
0    2.875472
1    2.950943
2    2.977778
Name: Shopping_Satisfaction, dtype: float64

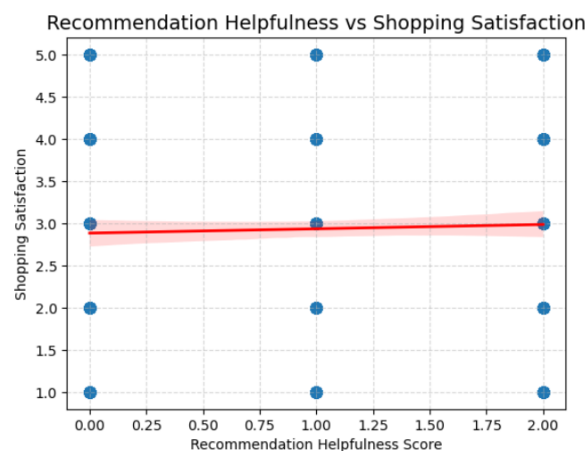
df['Recom_Helpfulness_Score'].corr(df['Shopping_Satisfaction'])

np.float64(0.029096160369359608)
```

--> The correlation between recommendation helpfulness and shopping satisfaction is very weak, indicating that recommendations currently have minimal impact on overall customer satisfaction

--> There is no significant linear relationship between recommendation helpfulness and shopping satisfaction.

```
sns.regplot(x='Recom_Helpfulness_Score', y='Shopping_Satisfaction', data=df, scatter_kws={'alpha':0.6, 's':60},
            line_kws={'color':'red', 'linewidth':2})
plt.title("Recommendation Helpfulness vs Shopping Satisfaction", fontsize=14)
plt.xlabel("Recommendation Helpfulness Score")
plt.ylabel("Shopping Satisfaction")
plt.grid(True, linestyle='--', alpha=0.5)
```



--> The regplot indicates no strong correlation between recommendation helpfulness and shopping satisfaction, as satisfaction levels are widely distributed across all levels of recommendation helpfulness. This suggests that other factors beyond recommendations may be influencing customer satisfaction.

Task 4.2: Evaluate how review reliability and helpfulness impact overall ratings.

- In this task, first, I have converted review reliability and review helpfulness into numerical values by mapping numeric values to string values.
- Created two new columns review the reliability score and review helpfulness score.

```
df['Review_Reliability'].value_counts()
```

```
Review_Reliability
Heavily      170
Rarely       165
Never        163
Moderately   152
Occasionally 150
Name: count, dtype: int64
```

```
df['Review_Helpfulness'].value_counts()
```

```
Review_Helpfulness
No      282
Sometimes 269
Yes     249
Name: count, dtype: int64
```

```
rely_mapping= ({ "Heavily": 4,"Moderately": 3,"Occasionally": 2,"Rarely": 1,"Never": 0})
df['Review_Reliability_Score'] =df['Review_Reliability'].map(rely_mapping)
df['Review_Reliability_Score'].value_counts()
```

```
Review_Reliability_Score
4      170
1      165
0      163
3      152
2      150
Name: count, dtype: int64
```

```
review_helpfulness_map= ({ "No": 0, "Yes":1, "Sometimes":2})
df['Review_Helpfulness_Score'] = df['Review_Helpfulness'].map(review_helpfulness_map)
df['Review_Helpfulness_Score'].value_counts()
```

```
Review_Helpfulness_Score
0      282
2      269
1      249
Name: count, dtype: int64
```

- Then grouped the data by rating score and calculated the means.

```
df.groupby('Review_Reliability')['Rating_Accuracy'].mean()
```

```
Review_Reliability
Heavily      3.076471
Moderately    2.940789
Never        3.085890
Occasionally  3.040000
Rarely       3.139394
Name: Rating_Accuracy, dtype: float64
```

```
df.groupby('Review_Helpfulness')['Rating_Accuracy'].mean()
```

```
Review_Helpfulness
No      3.131206
Sometimes 2.992565
Yes     3.048193
Name: Rating_Accuracy, dtype: float64
```

```
df[['Review_Reliability_Score', 'Review_Helpfulness_Score', 'Rating_Accuracy']].corr()
```

	Review_Reliability_Score	Review_Helpfulness_Score	Rating_Accuracy
Review_Reliability_Score	1.000000	-0.047201	-0.020609
Review_Helpfulness_Score	-0.047201	1.000000	-0.040033
Rating_Accuracy	-0.020609	-0.040033	1.000000

--> Review reliability and helpfulness have almost no impact on rating accuracy

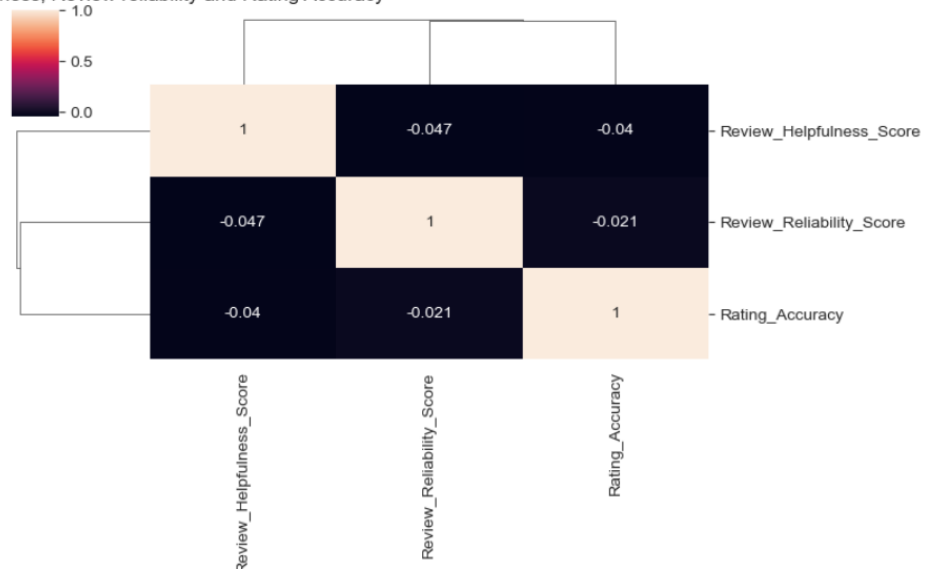
--> As reliability/helpfulness increases, Rating accuracy slightly decreases

- After identifying the correlation between review reliability score, review helpfulness score and Rating Accuracy.

```
sns.clustermap(df[['Review_Reliability_Score', 'Review_Helpfulness_Score', 'Rating_Accuracy']].corr(), figsize=(8, 5), annot=True)
plt.title('Correlation between Review Helpfulness, Review reliability and Rating Accuracy')
```

Text(0.5, 1.0, 'Correlation between Review Helpfulness, Review reliability and Rating Accuracy')

Correlation between Review Helpfulness, Review reliability and Rating Accuracy



--> This correlation indicates that there is weak relationship between variables, Also, These factors do not strongly influence each other. ¶

--> Customer perception of review reliability, helpfulness, and rating accuracy appears to be largely independent, indicating that improving one factor may not necessarily impact the others.

Task 4.3: Identify trends in how often customers engage with or trust personalised recommendations.

- For this task, I created a countplot for personalised recommendation frequency and taking recommendation helpfulness in hue. It tells whether customers who receive more recommendations actually trust and engage with them.

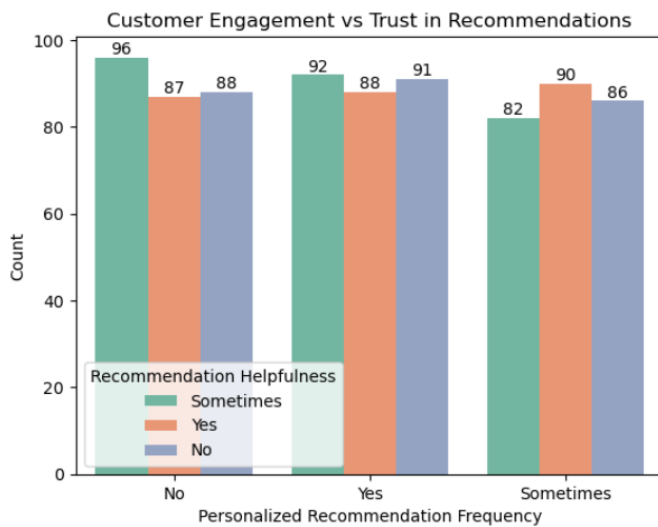
```
df['Personalized_recom_freq(cat)'].value_counts()
```

```
Personalized_recom_freq(cat)
No          271
Yes         271
Sometimes   258
Name: count, dtype: int64
```

```
df['Recom_Helpfulness'].value_counts()
```

```
Recom_Helpfulness
Sometimes   270
Yes         265
No          265
Name: count, dtype: int64
```

```
cpg = sns.countplot(x='Personalized_recom_freq(cat)', hue='Recom_Helpfulness', data=df, palette='Set2')
for container in cpg.containers:
    cpg.bar_label(container)
plt.title("Customer Engagement vs Trust in Recommendations")
plt.xlabel("Personalized Recommendation Frequency")
plt.ylabel("Count")
plt.legend(title="Recommendation Helpfulness")
plt.show()
```



--> It tells whether customers who receive more recommendations actually trust and engage with them.

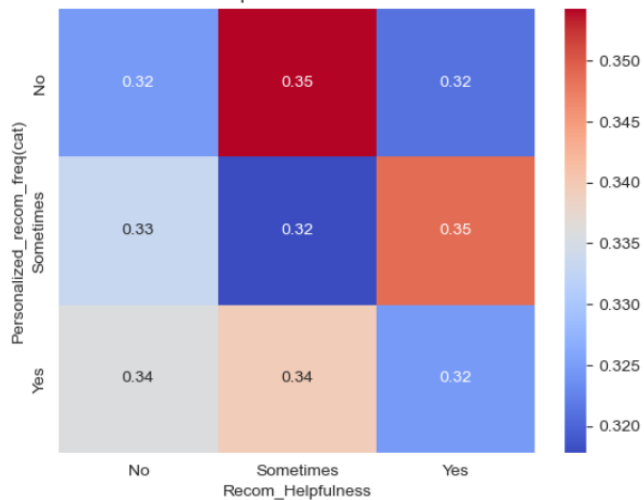
--> From the distribution, it is indicated that the personalised recommendation does not really influence the customer engagement and trust towards recommendations.

- For better visualisation of the correlation between both columns, I created a heatmap.

```
sns.heatmap(pd.crosstab(df['Personalized_recom_freq(cat)'],df['Recom_Helpfulness'],normalize='index'),
            annot=True,cmap='coolwarm')
plt.title('Correlation between Recommendation Helpfulness and Personalized Recommendation frequency')
```

Text(0.5, 1.0, 'Correlation between Recommendation Helpfulness and Personalized Recommendation frequency')

Correlation between Recommendation Helpfulness and Personalized Recommendation frequency



The analysis indicates that customer trust in personalized recommendations remains relatively consistent across different levels of engagement. Increasing the frequency of recommendations does not significantly enhance perceived helpfulness, suggesting that relevance and personalization quality are more critical than frequency.

Task 4.4: Suggest actionable insights for improving Target's recommendation system.

1. Customers don't find recommendations significantly helpful → low correlation

- The analysis indicates that personalized recommendations have a **minimal impact on customer satisfaction**, as reflected by the very weak correlation between recommendation helpfulness and overall shopping experience.
- This suggests that the current recommendation system is not effectively influencing customer perceptions.

- Actions: Use past purchase history,
Track browsing behavior
Recommend similar or complementary products
2. More Recommendations doesn't mean more trust
- Actions: Limit the number of recommendations per session
Show only top 3–5 highly relevant products

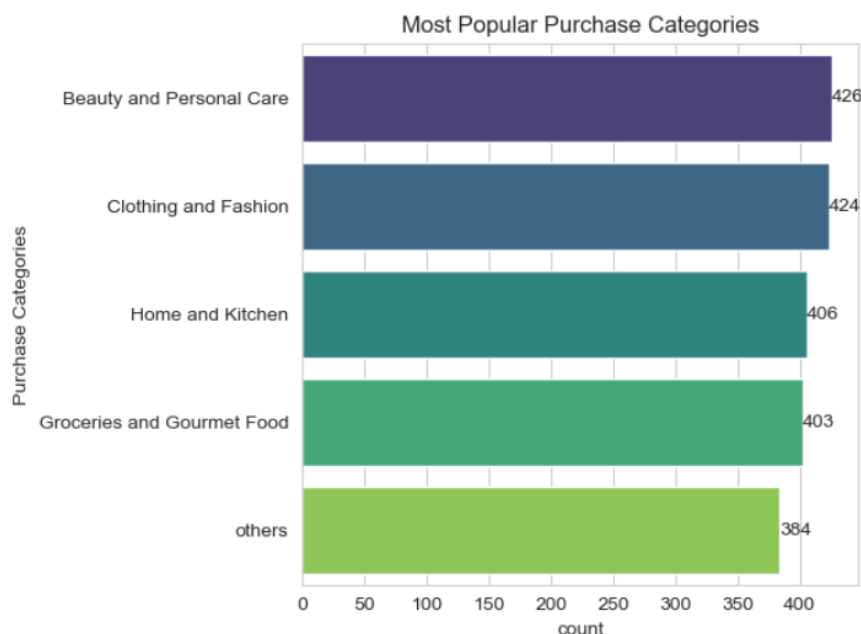
It will improve user experience and perceived usefulness

To enhance the effectiveness of the recommendation system, the following strategies are suggested:

- **Leverage behavioral data** such as past purchase history and browsing patterns to deliver more relevant and personalized recommendations.
- **Adopt a targeted approach** by recommending similar or complementary products aligned with customer preferences.
- **Limit the number of recommendations per session** to avoid overwhelming users, focusing instead on a curated set of high-quality suggestions (e.g., top 3–5 products).
- **Implement segment-based personalization**, tailoring recommendations according to different customer groups and behaviors.
- **Improve transparency and trust signals** by incorporating reliable reviews and clearly explaining why products are recommended.

Task 5: Visualization and Reporting

- **Purchase categories:** Created a countplot to see the distribution of the top-selling purchase category.

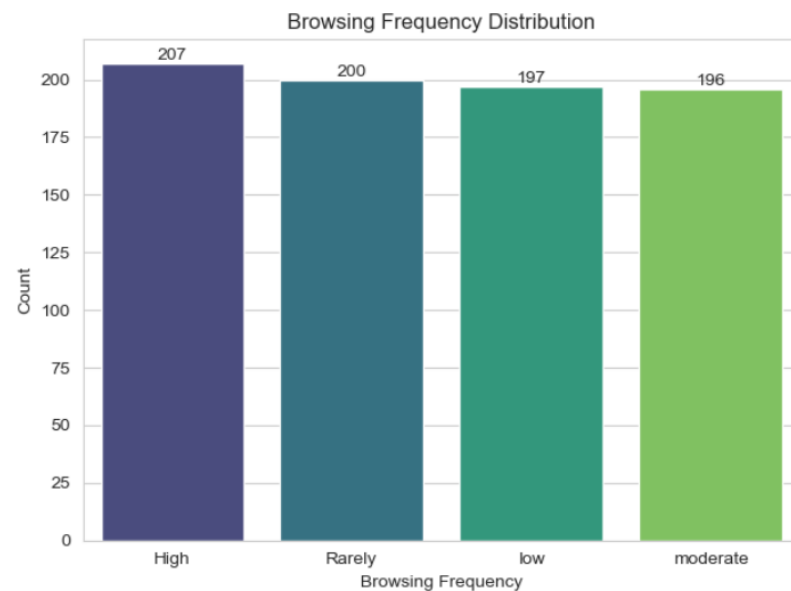


--> Most purchases are concentrated in 'Beauty and Personal Care' and 'Clothing and fashion' category

--> Indicates strong product demand concentration

--> Recommendation: Focus inventory on top-performing categories

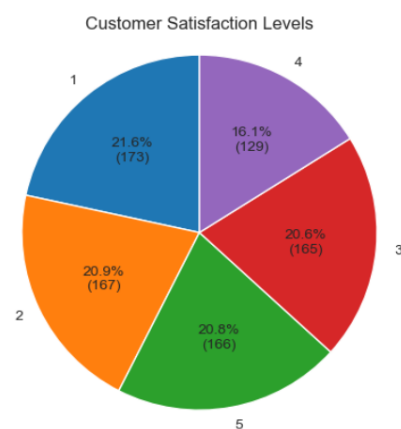
- **Browsing Frequency Distribution:** Created barplot firstly by aggregating the data in browse counts



- > Majority users browse Multiple times a day,
- > Shows active engagement but not always conversion
- > Recommendation: Improve conversion strategies

- **Satisfaction Levels :** I created a pie chart for shopping satisfaction of customers

```
counts = df['Shopping_Satisfaction'].value_counts()
def func(pct, allvalues):
    absolute = int(pct/100. * sum(allvalues))
    return f"{pct:.1f}%\n({absolute})"
plt.pie(counts, labels=counts.index, autopct=lambda pct: func(pct, counts), startangle=90)
plt.title("Customer Satisfaction Levels")
plt.axis('equal')
plt.show()
```



- > Most users fall in moderate to high satisfaction
- > Very few highly dissatisfied customers
- > Recommendation: Maintain service quality while improving weak areas

- **Correlation between recommendation usefulness and satisfaction:** for this I created a heatmap between Recommendation helpfulness and the satisfaction of the customers.

```
sns.heatmap(df[['Recom_Helpfulness_Score', 'Shopping_Satisfaction']].corr(), annot=True, cmap='Set2')  
plt.title("Correlation: Recommendation Helpfulness vs Satisfaction")  
plt.show()
```



--> There is a weak positive correlation (0.029) between recommendation helpfulness and shopping satisfaction.

--> This indicates that perceived usefulness or recommendation does not significantly influence customer satisfaction.

Recommendation: Focus on multiple factors such as service quality, pricing, and personalization to enhance overall user experience.

Task 6: Video presentation

Video link: <https://www.loom.com/share/8c805eb0d5f94f508559103e62579f9d>